Lecture 06 - L vs NL, BGS

Ah, I should say this. Next semester, I think, there is a course called "Seminar". I have prepared two topics for this course, which are quite advanced compared to the CC course, but which can eventually be extended to an MSc thesis, and for this there will exist scholarship money.

I will put the summary of the projects on the course's webpage. Or maybe talk about it at the end of today.

Review

We saw, so far:

- How one can encode computational problems in binary.
- How one can define, formally and rigorously, what an algorithm is.
- That some problems (e.g. halting) cannot be solved algorithmically (!).
- We saw three examples of computational resources (det. time, nondet time, and memory/space), and how one can measure how much resources an algorithm takes.
- We defined complexity classes based on these resources: P, NP, PSPACE, but also E, EXP, and we could have equally defined NE, NEXP, ESPACE, EXPSPACE.
- We saw that with more resources we can solve more problems (time/ndet-time/space hierarchy theorems).
- We saw how some problems are as hard as any other problem in a certain complexity class, and so these problems can be used as "representatives" of the difficulty of solving problems in that complexity class:
    - SAT, 3SAT, Hamiltonian Path, etc for NP
    - TQBF for PSPACE --- could have used Sokoban, generalized chess, etc

Today!

We will start looking at computational classes that are less powerful than P: L and NL.

We will talk about the relativization barrier: one of the first reasons why proving that P != NP is so hard to do.

Ok, then, what is L and NL?

We have defined PSPACE as the class of all problems that can be solved in polynomial space.

Let us, analogously, define L to be the class of problems that can be solved in O(log n) space.

First, let me convince you this makes some sense.

saw on the board: a schematic drawing of a deterministic logspace Turing machine

Definition. L is the class of computational problems which can be solved by deterministic logspace Turing machines.

Example of problems in L

- **Indexing**. Input is $i \in [n]$, $(x_1, \ldots x_n) \in \{0,1\}^n$, output x_i.
  - Any natural number between 0 and n - 1 can be encoded in binary using ... bits
- **Path Following.** Input is an n-vertex graph G, given by its adjacency matrix, and a sequence of vertices $x_1, \ldots, x_m$, each $x_i \in [n]$. The output is 1 if and only if the sequence of vertices forms a path, i.e., all edges (x_i, x_{i+1}) are present.
- **Product of permutations.** (Homework) Input are m permutations of $[n]$, $(\sigma_1, \ldots, \sigma_m)$. Output is their product $\sigma_1 \cdot \cdots \cdot \sigma_m$
- **REALLY FUCKING NOT OBVIOUS:** Undirected connectivity is in L (Reingold 2008). I.e..
  - Input: An undirected graph G on n vertices, and two vertices s, t in $[n]$.
  - Output: Is there a path between s and t in G?

  I want you to think deeply about this problem for a while. You are given the adjacency matrix of G. This can be a HUGE HUGE HUGE object compared to the amount of memory that you have. You only have O(log n) bits of memory. So, e.g., if you have 200 bits of memory, that would make the encoding of the graph that you are given larger than the known universe.

  Now you have to navigate this map. Your memory only allows you to know where you are, and maybe what neighbours are adjacent to you, and little more.

  You start at s, say. You have to know if there exists a path to t. How on earth do you do this?

  It's like walking blind. And yet, it can be done
- How about **directed** graphs?! It turns out that the problem here is much harder. And the reason has to do with the existence of examples as I will now draw on the board.

  If I take a directed line 1 -> 2 -> ... -> n, and put edges backwards from every vertex i to 1, then a random walk starting at 1 will take exponential time to reach n. In the same example if the graph is undirected, it only takes poly(n) time to reach n. This is at the core of why directed connectivity, which is also known as reachability, is such a hard problem.

  Input: A *directed* graph G, two vertices s and t

  Output: Does there exist a (directed) path from s to t?
- Notice that the problem seems hard to solve in logspace, but it is easy to verify, in logspace, whether a given directed path is actually present in the graph. Sound familiar? (hard to solve, easy to verify given a witness?)
- So let us look at *non-deterministic* Turing machines working in logspace. This defines the class NL, which is a class of decision problems.
- Claim: Reachability is in NL.
  - why? To check if there exists a path between s and t, one simply "guesses" the sequence of nodes along the path, and verifies that each successive pair of guessed nodes has a directed edge between them.
- Alternative definition of NL: (in board, using verifier with one-way tape)
  - We let NL be the class of problems A for which there exists a logspace Turing machine M in L such that

    $$x \in A \iff \exists y \in \{0,1\}^{n^c} M(x,y) = 1$$

    just like NP, except that the certificate y can only be accessed by a one-way tape.

- Another big unsolved question in computer science: Is $NL \subseteq L$?

  It turns out that **directed reachability is NL-complete!**

  So the L vs NL problem can be phrased in other words, as: is connectivity as easy to solve in directed graphs as it is in undirected graphs?

  If you think about it for a while, you may realize that this is a really basic and profound question.

- Now let me show to you that reachability is NL-complete, and then we move onto other things.
- As a side node (this is homework), it turns out that NL = coNL! This is really not obvious, and the analogous question higher up, whether NP = coNP, is a very profound question in its own right ("if there are short proofs that something is true, must there also exist short proofs that it is false").

  But the NL vs coNL problem has been solved.

- So now for the proof that reachability is NL-complete. To prove this, we need a notion of reduction that makes sense for the L vs NL problem.

Definition. A function $f : \{0,1\}^* \to \{0,1\}^*$ is computable in logspace if there exists a deterministic logspace turing machine that on input x outputs $f(x)$ (on the write-only output tape).

Definition. A problem $A \subseteq \{0,1\}^*$ is called *NL-hard* if for every problem B in NL there exists a logspace-computable function f such that

$$x \in B \iff f(x) \in A$$

We then write $B \leq_L A$. If A is in NL and is NL-hard, we call it NL-complete.

Theorem. Reachability is NL-complete.

Proof. Let B be any problem in NL. Let M be a non-deterministic Turing machine that solves B using $c \log n$ space, for some constant c. Meaning, $x \in B$ if and only if there exists a sequence of applications of M's transition functions causing $M(x)$ to accept.

Now let's associate with M and x the *graph of configurations* $G_{M,x}$ of M on input x. The nodes in this graph are all possible configurations of M that use $c \log n$ space (here we don't count the contents of the input tape, but we do count the position of the head of the input tape). We connect configuration C_1 with configuration C_2 by a directed edge, if, being at configuration C_1, with input x, at least one of the two possible transitions goes to configuration C_2.

Then $x \in B$ iff there exists a path from the initial configuration s to some accepting configuration. We can add a dummy node t to the graph, and put an edge from every possible accepting configuration to that node. Then $x \in B$ iff there exists an s to t path in the configuration graph.

We only need to show that the function f which accepts input x and outputs the above graph, and the two nodes s and t, can be computed in logspace. Then it will hold that
$x \in B \iff f(x) \in REACHABILITY$

This is not too hard to see. Note that the configurations only take O(log n) space, so a deterministic logspace turing machine can run over every pair (C_1, C_2) of configurations (simply by enumerating them). For each such pair, it simulates the execution of M on C_1, with both transition functions, and sees if the result of applying one of them happens to be C_2. In that case, it outputs 1 (the (C_1, C_2) edge is present in the graph), otherwise, it outputs 0.

So, undirected vs directed connectivity = L vs NL.

Homework: show that if $A \in NL$ and $B \leq_L A$, then $B \in NL$.

(i.e. NL is closed under logspace reductions; so if A is "not-harder-than-NL", and B <= A, then B is also not-harder-than-NL.)

Part 2: the BGS theorem

OK, good!

We are almost done with Part 1 of the course. This is what I would call "structural" computational complexity. We are interested in complexity classes, and computational problems, and about their relative difficulty. It is a theory of "how hard are computational problems, relative to each other."

But how about the absolute questions? I don't care if P = NP iff SAT in P, I just care to know the damned answer: is P = NP, or not?!

This is a hard question for several reasons. We will now see one of them.

Note that, so far, the only "separations" between complexity classes that we have seen, are all proven by diagonalization. This was true for the halting problem, which we showed is not computable. This is also true for the time hiearchy, non-det-time-hierarchy and space-hierarchy theorems.

The arguments that work via diagonalization always have the same outline:

one proves that a computational complexity class C has a problem that cannot be solved in class A by showing that C can "simulate" all algorithms of A, and do something different to all of them.

So for example, in the time-hiearchy theorem:

in time g(n) which is much bigger than f(n), I can simulate any machine that only runs in time f, and do the opposite of what it does on some input. By construction, this behavior cannot be done in time f.

So, what are the chances that P vs NP can be shown via such an argument?

Maybe in NP we could "simulate all problems in P" or something like that.

Actually, we can prove that there is no way that this is going to work.

The argument against such a proof goes more or less like this.

- First we need to discuss what a "relativized computation" is.
    - Suppose that you are writing an algorithm that can invoke a special subroutine, called an "oracle", that solves some computational problem.
    - Formally one defines it as follows.
    - Let $A \subseteq \{0,1\}^*$ be any set, which we will call "the oracle".
    - The Turing machine has a special tape, called the "oracle tape".
    - The machine can, at any point in the computation, write down some string y in the oracle tape, enter a special state ("press a button"), and the string y is erased, and replaced with a bit, indicating whether $y \in A$ or $y \notin A$.

- Now we make the following observation: All of the diagonalization arguments work relative to any oracle.
- Meaning, we have shown, for example, that $P \neq E$. It turns out that the very same proof shows that, for any oracle A, $P^A \neq E^A$. Meaning, in exponential time with oracle A I can solve more problems than in polynomial time with oracle A.
- This can be seen by looking at the proof. We show that E^A contains a problem not in P^A by simulating every possible polynomial-time-with-oracle-A computation and doing the opposite on some input. This is possible in the same way as before: when we are simulating a machine and it makes an oracle query, the simulator also makes the same query, and provides the answer to the machine being simulated.
- Good. More generally speaking, every diagonalization argument known has this feature: it also works relative to any oracle A.

However, in 1985, Baker, Gill and Solovay made the following observation:

BGS'85. There exists an oracle A such that $P^A = NP^A$, and another oracle B such that $P^B \neq NP^B$.

This means that any proof technique that "relativizes", meaning, it remains true for any oracle, will necessarily fail to prove that $P \neq NP$, or that $P = NP$. So it cannot settle the question.

Proof. We let $A = TQBF$. It turns out that $P^A = NP^A = PSPACE$.

Now to construct B. We do it in stages. The idea is the following.

At each length n, we will choose a single string $x \in \{0,1\}^n$, and sometimes include it in B, and sometimes not. We will show that we can choose x in such a way that it is "hidden" from all polynomial-time computations, meaning, every polynomial-time computation will not query x, so it can't tell whether $x \in A$ or not. But a non-deterministic Turing machine with oracle A can simply guess some string y of length n and accept iff $y \in A$.

Ok, this is how we build B.

We start by letting $B = \varnothing$ be the empty set, and add strings to it one at a time, so that the following non-deterministic machine will behave differently than any polynomial-time machine:

```
Algo() // Non-deterministic-algorithm with oracle B
  on input 0^n
  y = guess a string of length n using non-determinism
  output B(y)
```

First, observe that: Algo^B(0^n) = 1 if and only if there exists y of length n in B.

Now this is how we build B:

- Let $B_0 = \varnothing$
- Let us take an enumeration of all possible oracle Turing machines $M_1, M_2, \ldots$
- For each $n = 1, 2, \ldots$:
  - Run $M_n(0^n)$ with oracle B_{n-1}, as it was constructed so far, for a maximum of $2^{n/10}$ time. Let Q be the set of queries that are made by any the computations $M_1(0^1), M_2(0^2)$ *etc* that we ran so far.

    - Claim: $|Q| \leq n2^{n/10}$, just because one computation can do at most one query per time step.
    - Hence there must exist some $x_n \in \{0,1\}^n$ which is not in Q.
    - We then add x_n to B if and only if $M_n(0^n) = 0$.
    - It follows that $Algo^B(0^n)$ does the exact opposite of $M_n(0^n)$.
    - Crucially, it also follows that $M_n(0^n) = M_n^B(0^n)$, i.e., the behavior of the machines does not change when given oracle B, because we have always added strings to B that were not queried in any of the computations of $M_n(0^n)$.
    - It then follows that $Algo^B(0^n) \neq M_n^B(0^n)$, for all n.
    - So $NP^B \neq P^B$.
- We are exploiting the fact that, in some operational sense, non-deterministic machines *can* do more than polynomial time machines. Namely they can do the OR of an exponential number of queries to the oracle B.
- However, so far, there appears to be no way of coming up with such an argument without the oracle.

Conclusion: P vs NP cannot be settled by relativizing techniques.

When this was proven, in 1984, almost all of the known computational complexity separations were proven via relativizing techniques!

Some wild conjectures were made back then, e.g. that two complexity classes C_1 C_2 such that $C_1^A \neq C_2^A$ for a "random oracle" A would actually have to be not equal without an oracle. (it is possible to show that $P^A \neq NP^A$ relative to a random oracle whp).

But then this turned out to be false (IP = PSPACE, but $IP^A \neq PSPACE^A$ relative to a random oracle)

(((

What is IP?

Suppose that there are two entities, Merlin and Arthur.

Merlin is all powerful, he can do any computation he wants, answer the halting set, anything.

Arthur is "dumb" like us, he is a polynomial time dude.

Merlin wants to convince Arthur that a certain statement is true. E.g. that $x \in B$.

But Arthur doesn't trust Merlin. Maybe he isn't actually speaking to Merlin, but instead he is speaking to a witch that disguised herself as merlin.

So, we define the notion of interactive proof as follows. A set $B \in IP$ if and only if there exists a randomized polynomial-time "protocol" A (a protocol is an algorithm that interacts with another agent).

The properties that A must have are as follows:

- Completeness. If $x \in B$ is indeed true, there exists a way of interacting with A that will cause A to output 1 with high probability.

- Soundness. But if $x \notin B$, then no matter how one interacts with arthur, no matter what we say to him (and really we can uses ANY strategy including computationally unbounded strategies, so we are like GOD), the probability that he is convinced is low.
- It turns out, IP = PSPACE!

We can also consider proofs where the interaction is limited, e.g. Arthur asks a question, then Merlin (or the witch) answers, then Arthur decides based on the answer. This class is called AM ("arthur-merlin"), but we could also consider AMAM (arthur-merlin-arthur-merlin) and so on.

These classes interlace the polynomial-time hierarchy.

These notions of proofs might appear quite abstract, but actually, they have recently found application in cryptosystems.

For example there is this crypto coin "Zcash". The statements that one needs to be convinced are all of the form "wallet X has transferred z ammount of money to wallet Y" and similar statements. The deal here is that one should be convinced of this *without* known how much money X has in total, or who has transferred money to X in the past, and so on, so that the system remains anonymous.

This requires a new notion of interactive proof, called a "zero-knowledge proof", also outside of the scope of this course.

Is there a relation with classical proofs?

Well, there is "a relation", yes. E.g. NP is in AM. It is an open problem, however, whether AM is contained in NP.

)))

So diagonalization and relativizing techniques won't settle whether P = NP.

Then, what happened next?

The field turned more combinatorial. We started studying circuits, branching programs, and other kinds of algorithms, which are represented by combinatorial objects.

Such computational models cannot be given "an oracle", because their combinatorial behavior changes drastically when this is done. So there was hope that combinatorial analysis could lead to an answer to the P vs NP question and related questions.

In part 2 of the course, we are going to study some of the results that came in this era. You will see that they are heavily combinatorial (probabilistic method, graphs, sunflower lemma, etc)

If we manage, at the very end of the course, we will see why these combinatorial approaches are also unlikely to solve the P vs NP problem.

This is the natural proofs barrier, and it is the most mighty obstacle known, against solving the P vs NP problem.